MSME Credit Pipeline Scheduling

ScoreMe Advanced Systems Design Assessment

Gaurav Lodhi

October 29, 2026

Conventions

Slots are 0-based, $\sigma(t_i) \in \{0, \ldots, K-1\}$, exactly as the provided generator produces windows. The base penalty therefore charges $\sigma(t_i)+1$, which equals the spec's 1-based slot index: slot 0 still costs w_i, and the optimum is never 0. The toy instance of §3.3 is stored with its windows shifted by -1. We write $W = \sum_i w_i$.

1 Task 1: NP-Hardness

We reduce from Graph 3-Colouring: given $G = (V, E)$, is there a $\chi : V \to \{0, 1, 2\}$ with $\chi(u) \neq \chi(v)$ for every $\{u, v\} \in E$? It is NP-complete (Karp, 1972). Our target is the decision problem Sched-Feas: does an instance (G', r, C, w, τ) have an assignment satisfying F1, F2 and F3? The construction uses only the generator's own form: $d = 4$, uniform capacity $C = (32, 128, 8, 6)$ in every slot, and SLA windows with $l < u$. So hardness holds for the generator's own instance class, not just for a generalisation of it.

1.1 Construction $f(G)$

Let $m = |V|$. Output the following instance:

- $K = 5$ slots, each with capacity C.
- One *vertex task* t_v for each $v \in V$, with $r(t_v) = C/(m+1)$ (componentwise) and window $\tau(t_v) = [0, 3]$.
- One *blocker task* b with $r(b) = C$ and window $\tau(b) = [0, 3]$.
- Conflict graph $E' = \{(t_u, t_v) : \{u, v\} \in E\}$. The blocker has no conflicts.
- Weights $w \equiv 1$. Any weights work, because feasibility does not depend on them.

The output has $m+1$ tasks, $|E|$ conflict edges and constant K and d, so f runs in $O(m+|E|)$ time.

The role of each family. The three gadgets together reduce the available colours from 5 slots to exactly 3:

- *SLA windows (F3)* restrict every task to slots 0..3. Slot 4 exists but no task may use it.
- *Capacity (F2)*: b fills its slot completely, in every dimension. Every vertex task has strictly positive demand, so no vertex task fits next to b. That removes one more slot.
- *Conflicts (F1)* encode E on the 3 slots that remain.

1.2 Correctness

Theorem 1. *G is 3-colourable $\iff$ $f(G)$ has an assignment satisfying F1, F2 and F3.*

Proof. ($\Rightarrow$, a colouring gives a schedule.) Let χ be a proper 3-colouring. Set $\sigma(t_v) = \chi(v) \in \{0, 1, 2\}$ and $\sigma(b) = 3$. **F3:** every slot used lies in $[0, 3]$. **F1:** a conflict edge (t_u, t_v) comes from $\{u, v\} \in E$, so $\sigma(t_u) = \chi(u) \neq \chi(v) = \sigma(t_v)$. **F2:** slot 3 holds only b, with load exactly C. Each slot $s \leq 2$ holds at most m vertex tasks, with load at most $\frac{m}{m+1}C < C$. Slot 4 is empty.

($\Leftarrow$, a schedule gives a colouring.) Let σ satisfy F1, F2 and F3, and let $s_b = \sigma(b)$. By F3, $s_b \in \{0, 1, 2, 3\}$, and every vertex task lies in $\{0, 1, 2, 3\}$. By F2, no vertex task shares s_b: in any dimension k that load would be $C_k + C_k/(m + 1) > C_k$. So every vertex task uses one of the three slots $F = \{0, 1, 2, 3\} \setminus \{s_b\}$. Let $\pi : F \to \{0, 1, 2\}$ be the order-preserving bijection, and set $\chi(v) = \pi(\sigma(t_v))$. For $\{u, v\} \in E$, F1 gives $\sigma(t_u) \neq \sigma(t_v)$, so $\chi(u) \neq \chi(v)$. Hence χ is a proper 3-colouring. □

Theorem 2. SCHED-FEAS *is NP-complete. Minimising* any *P subject to F1–F3, including ours, is NP-hard.*

Proof. It is in NP: a certificate σ is checked in $O(nd+|E|+Kd)$ time (this is exactly `solver/validate.py`). Hardness follows from the theorem above and the polynomial construction. An optimiser must either output a feasible σ or correctly report that none exists, so it also decides SCHED-FEAS. □

Consequence used in §4. Unless P = NP, no polynomial algorithm, SSD-R included, can find a feasible assignment whenever one exists. For the same reason, no polynomial algorithm can have a finite approximation ratio on all instances. Task 4's guarantees are therefore stated for the instances on which SSD-R returns a schedule, and for explicit tractable sub-classes.

1.3 Every family is necessary

If any one gadget is removed, the equivalence fails. K_4, which is not 3-colourable, then maps to a *feasible* instance:

gadget removed	what breaks
F1 (no conflicts)	every instance is feasible: put all vertex tasks in one free slot
F2 ($r(b) = 0$)	b shares a slot, so the vertex tasks get 4 colours {0..3}
F3 (windows $[0, 4]$)	b moves to slot 4, so the vertex tasks get 4 colours {0..3}

So the reduction is a genuine compound of colouring, packing and temporal constraints, not colouring in disguise. In particular, F2 and F3 each remove exactly one colour.

1.4 Machine check

`tools/reduction.py` builds $f(G)$ and solves it exactly with `solver/bruteforce.py`. It compares the result with an independent 3^m colouring check, and extracts χ from σ as in the ($\Leftarrow$) proof:

G	3-colourable	$f(G)$ feasible	extracted χ proper
C_5	yes	yes	yes
K_4	no	no	–
W_5 (wheel)	no	no	–
Petersen	yes	yes	yes

For each of the three ablations, K_4 becomes feasible, which confirms the table above. Both checks are unit tests (`tests/test_reduction.py`).

2 Task 2: Penalty Function $P(\sigma)$

2.1 Definition

For a complete assignment σ, let $\ell_{s,k}(\sigma) = \sum_{i:\sigma(t_i)=s} r_k(t_i)$ be the load of dimension k in slot s, and $u_{s,k} = \ell_{s,k}/C_k(s)$ its utilisation. Let $\mathcal{C}_k = \{s : C_k(s) > 0\}$ be the slots that have capacity in dimension k.

$$P(\sigma) = P_{\text{base}}(\sigma) + \lambda_R\, R(\sigma) + \lambda_B\, B(\sigma), \qquad \lambda_R = \lambda_B = 1$$

$$P_{\text{base}}(\sigma) = \sum_i w_i\,(\sigma(t_i) + 1)$$

$$R(\sigma) = \sum_{i:\, u_i > l_i} w_i \left(\frac{\sigma(t_i) - l_i}{u_i - l_i}\right)^2 \qquad \text{(SLA breach risk)}$$

$$B(\sigma) = W \cdot \frac{1}{\sum_k |\mathcal{C}_k|} \sum_{k=1}^{d} \sum_{s \in \mathcal{C}_k} (u_{s,k} - \bar{u}_k)^2\,, \quad \bar{u}_k = \frac{1}{|\mathcal{C}_k|} \sum_{s \in \mathcal{C}_k} u_{s,k} \qquad \text{(load imbalance)}$$

Here $[l_i, u_i]$ is the SLA window of t_i; the u_i in R is the window end, not a utilisation.

2.2 Motivation in the ScoreMe context

SLA breach risk R. A bureau pull placed in the last slot of its window has no slack left. A single retry against a slow bureau API, or a delayed upstream OCR job, then breaches the lender SLA. R measures where each task sits *inside its own window*: 0 at the window start, rising to w_i at the window end. It is squared because risk is concentrated near the deadline. Moving a task from the middle of its window to the end costs three times as much as moving it from the start to the middle.

Load imbalance B. If one slot runs at 100% GPU while the next runs at 10%, the OCR cluster has no headroom in the busy slot for retries or late-arriving jobs, and the idle slot wastes capacity that has already been paid for. B is the mean squared deviation of utilisation from its per-dimension mean, scaled by W so that it is in the same units as P_{base}. A slot with zero capacity in some dimension can hold no load in it. Its utilisation would be $0/0$, so it is left out of that dimension. Counting it as 0% would push work into a slot that cannot take any.

2.3 Required properties

1. **Polynomial time.** P_{base} and R take one pass over the tasks, $O(n)$. B builds the load table in $O(nd)$ and then reads it in $O(Kd)$. In total, P takes $O(nd + Kd)$ time.

2. **Monotonically meaningful.** Each term is non-negative and falls exactly when the matching operational quantity improves. P_{base} falls when weighted delay falls. R falls when tasks move away from their deadlines. B is 0 if and only if every dimension is equally utilised in every usable slot.

3. **Non-trivial.** The extension is not a constant, and it is not a rescaling of P_{base}. Take two unit tasks A with window $[0, 1]$ and B with window $[0, 2]$, and $K = 3$. The assignments $(A{\to}1, B{\to}0)$ and $(A{\to}0, B{\to}1)$ both have $P_{\text{base}} = 3$. But $R = 1$ for the first, because A is at its deadline, and $R = 1/4$ for the second. So P prefers the second, which P_{base} cannot tell apart (`test_extension_is_not_trivial`).

2.4 Bound used in Task 4

Lemma 3. *For every feasible σ: $P_{base}(\sigma) \le P(\sigma) \le \beta\, P_{base}(\sigma)$, where $\beta = 1 + \lambda_R + \lambda_B/4 = 2.25$.*

Proof. $R, B \ge 0$ gives the lower bound. Each ratio in R lies in $[0, 1]$, so $R \le W$. Feasibility (F2) gives $u_{s,k} \in [0, 1]$, and the variance of values in $[0, 1]$ is at most $1/4$, so $B \le W/4$. Finally, $\sigma(t_i) + 1 \ge 1$ gives $W \le P_{\text{base}}(\sigma)$. Hence $P \le P_{\text{base}} + \lambda_R W + \lambda_B W/4 \le \beta\, P_{\text{base}}$. □

This lemma lets an approximation ratio for P_{base} carry over to P with an extra factor of at most β (Task 4). It also makes the brute-force bound valid for P. The bound $\text{cost} + \sum_{\text{unassigned}} w_i(l_i + 1)$ is a lower bound on P_{base}, and therefore on P.

2.5 Worked example (toy instance)

For $\sigma = (T1{\to}1, T2{\to}0, T3{\to}0, T4{\to}1, T5{\to}1, T6{\to}2)$, which is the exact optimum: $P_{\text{base}} = 33$, and $R = 5 \cdot \frac{1}{4} + 3 \cdot \frac{1}{9} + 2 \cdot \frac{1}{4} = 2.0833$. The utilisation vectors over slots 0..3 are: CPU and RAM $(0.1875, 1, 0.125, 0)$; GPU $(0, 1, 0, 0)$; Net $(0.833, 0.5, 0.25, 0)$. They give $B = 19 \cdot 2.3704/16 = 2.8149$, so $P = 37.898$. These values are asserted in `tests/test_penalty.py`.

2.6 Observation

On the three small benchmarks (seeds 1, 2, 3), the exact optimum under P is the *same* assignment as the optimum under P_{base}. The extension adds 9–15% to the value but did not change the best schedule. With $\lambda_R = \lambda_B = 1$, delay dominates, and R and B mostly break ties between near-equal schedules. A platform that treats SLA risk as more important would raise λ_R. The bound in Lemma 3 then grows to $\beta = 1 + \lambda_R + \lambda_B/4$.

3 Task 3: Algorithm Design — SSD-R

SSD-R: SLA-Slack DSATUR with Certified infeasibility and Repair. SSD-R is a hybrid with four stages. Stage 1 certifies infeasibility using necessary conditions. Stage 2 is a saturation-ordered constructive greedy (DSATUR generalised to windows and capacity) with priority restarts. Stage 3 is a one-level eviction repair. Stage 4 is a local search on P using moves and swaps. Labels Lx and Rx below appear as comments in `solver/algorithm.py`, so the code can be matched to the pseudocode line by line.

3.1 Pseudocode

Notation: $N(i)$ is the set of conflict neighbours of t_i, and $\text{load}(s) \in \mathbb{R}^d$ is the current load of slot s. $F(i) = \{s \in [l_i, u_i] : N(i) \cap \sigma^{-1}(s) = \emptyset,\ \text{load}(s) + r_i \le C(s)\}$ is the set of slots that are currently feasible for t_i. $c(i, s) = w_i(s + 1) + \lambda_R w_i \big(\frac{s - l_i}{u_i - l_i}\big)^2$ is the separable part of P, and ΔB is the change in the imbalance term.

3.2 Line-level justification

L1, L3 Each is a necessary condition. If one fails, the failure is a checkable proof of infeasibility (Lemma 5). L3 covers every interval, not only $[0, K - 1]$. Tasks with narrow windows can overload a sub-range even when the global totals fit.

L4 Tasks confined to $[a, b]$ must be coloured with $b - a + 1$ colours. For width 2 this is exactly bipartiteness, which BFS decides in linear time, so the check is *exact*. Across the 9 benchmarks, 31% of windows have width 2, and 50% have width at most 3, so this case matters most in practice. For width ≥ 3, k-colourability is NP-hard. A clique larger than the width is a sound but incomplete certificate; it is grown greedily from every start vertex.

L8 This is the core DSATUR idea, adapted to this problem. A task's "saturation" is the number of slots *still* open to it, which shrinks through conflicts (F1), capacity (F2) *and* the window (F3). The most constrained task is placed first, while it still has options. Ties go to higher degree, because that task constrains the most others, then to heavier weight, because it should get the earlier slot. Index breaks the final tie, so runs are deterministic.

Algorithm 1 SSD-R(T, G, r, C, w, τ)

Stage 1 — certify infeasibility

L1: **for** each t_i: **if** no $s \in [l_i, u_i]$ has $r_i \le C(s)$ **return** INFEASIBLE ▷ fits nowhere

L2: **for** each interval $[a, b] \subseteq [0, K-1]$: $S \leftarrow \{i : a \le l_i,\ u_i \le b\}$

L3: **if** $\exists k : \sum_{i \in S} r_{ik} > \sum_{s=a}^{b} C_k(s)$ **return** INFEASIBLE ▷ interval demand

L4: **if** $G[S]$ is visibly not $(b{-}a{+}1)$-colourable **return** INFEASIBLE

(width 1: any edge; width 2: odd cycle by BFS; width ≥ 3: greedy clique $> b{-}a{+}1$)

Stage 2 — construct, with restarts

R1: $\Pi \leftarrow [\,]$; **repeat** at most n times: ▷ Π: promoted tasks

R2: run L5–L11 with promotion list Π; **if** it succeeds **goto** L18

R3: move the failed task to the front of Π

return NOT_FOUND (best partial σ)

L5: $U \leftarrow T$, $\sigma \leftarrow \emptyset$

L6: **while** $U \neq \emptyset$:

L7: compute $F(i)$ for all $i \in U$

L8: $i^* \leftarrow \arg\min_{i \in U} \big(\mathrm{rank}_\Pi(i),\ |F(i)|,\ -\deg(i),\ -w_i,\ i\big)$

L9: **if** $F(i^*) = \emptyset$: $F(i^*) \leftarrow$ REPAIR(i^*); **if** still $\emptyset$ **fail with** i^*

L10: $S \leftarrow \{s \in F(i^*) : \nexists j \in U \setminus \{i^*\}$ with $F(j) = \{s\}$ that i^* would block$\}$, or $F(i^*)$ if empty

$\sigma(i^*) \leftarrow \arg\min_{s \in S} \big(c(i^*, s) + \lambda_B \Delta B,\ s\big)$ ▷ forward check, then cheapest

L11: $U \leftarrow U \setminus \{i^*\}$

Stage 3 — REPAIR(i)

L12: **for** $s = l_i, \ldots, u_i$:

L13: $B \leftarrow N(i) \cap \sigma^{-1}(s)$ ▷ conflict blockers

L14: **while** r_i does not fit in s without B: add to B the largest task of s ▷ overflowing dim

L15: tentatively move each $j \in B$ to its cheapest feasible slot $\neq s$

L16: **if** all moved and $s \in F(i)$ **return** $\{s\}$ **else** roll back

L17: **return** $\emptyset$

Stage 4 — improve

L18: **repeat** until no improvement (at most 50 passes):

L19: for each i and $s \in [l_i, u_i]$: move $i \to s$ if feasible and $\Delta P < -\varepsilon$

L20: for each pair i, j with $\sigma(i) \neq \sigma(j)$: swap if feasible and $\Delta P < -\varepsilon$

L21: **return** σ

L10 First a forward check: a slot is skipped if it is the *last* feasible slot of another unplaced task j, and i^* would block it there (a conflict with j, or no capacity left for both). Among the remaining slots, the task takes the one that adds the least to P. Without the forward check, the cheapest-slot rule repeatedly walked into dead ends. On $(100, 10, 0.05)$ seed 1, four tasks with nested windows evicted each other in a cycle through all n restarts (§6). The tie rule "lower slot" keeps zero-weight tasks deterministic (used in the tight example, §4).

L14 If capacity, not a conflict, is what blocks i, the largest task in the overflowing dimension is evicted first, so that as few tasks as possible move.

L15–L16 Repair goes only one level deep: evicted tasks must go to slots that are already feasible for them. Chains of evictions would turn repair into a full search. All moves are rolled back on failure, so the partial assignment stays feasible at all times.

R1–R3 Squeaky-wheel restarts. A task that fails late was usually squeezed out by earlier choices, so it is promoted to go first in the next pass. On 1200 random small instances, restarts cut the feasible instances that SSD-R missed from 58 to 2 (§6).

L19–L20 A move lowers P directly. A swap fixes cases that no single move can, for example when a full slot holds a light task and a heavy task waits in a later slot. Only strict improvements are accepted, so P falls at every step and the search cannot cycle.

3.3 Complexity

Stage 1 checks $O(K^2)$ intervals. Each costs $O(nd)$ for demand and $O(n^2\omega)$ for the clique, where ω is the clique size. One pass of L5–L11 does n iterations, each computing F for at most n tasks in $O(Kd)$ using a per-(task, slot) conflict counter. Repair costs $O(K \cdot n \cdot Kd)$. With at most n restarts, Stage 2 is $O(n^3Kd + n^2K^2d)$. One local-search pass is $O(n^2d)$, with at most 50 passes. Every term is polynomial in n, K, d. Measured runtimes are in §6.

3.4 Design rationale

The problem is DSATUR-shaped because, on generator instances, the binding constraint is almost always the *interaction of narrow SLA windows with conflicts*. Most windows span only 2–3 slots, so a task with a few neighbours already in its window has no slot left. A greedy that ignores how many options each task has left (for example, heaviest weight first) places flexible tasks first and strands the constrained ones. Saturation ordering places each task while it still has a choice.

Stage 1 comes first because infeasibility is *common* on these instances. Six of the nine graded benchmarks are infeasible (§6). A heuristic that only reports "not found" cannot tell a hard instance from an impossible one. Certificates make that distinction and give the operator an actionable reason, such as "these 5 bureau pulls form an odd conflict cycle inside a 2-slot SLA window".

Penalty-awareness appears in two places. L10 is a myopic cost-aware choice. Stage 4 recovers the global trade-offs, such as delay against SLA risk against load balance, that a one-pass greedy cannot see.

3.5 Alternatives considered and rejected

1. **Exact branch-and-bound (or an ILP).** This gives the optimal σ, and the infeasibility proof is exhaustive. We implemented it (`solver/bruteforce.py`) and use it as ground truth. It was rejected as the main algorithm because it is exponential: K^n leaves in the worst case, which is 20^{200} for the sparse stress instance. Writing it as an ILP would also need a forbidden solver.

2. **Simulated annealing on a relaxed objective** $P(\sigma) + M \cdot \#\text{violations}$. It is easy to write, and the penalty handles all three constraint families the same way. It was rejected for three reasons. (i) It can never *prove* infeasibility. On the 6 infeasible benchmarks it would run to its time limit and report "best effort, still violating", with no reason. (ii) It is randomised, so a viva trace or a production incident cannot be reproduced without the seed, and its quality depends on the cooling schedule, M and the iteration budget. (iii) It has no worst-case structure from which an approximation ratio can be derived.

4 Task 4: Guarantees, Approximation Ratio and Tight Example

Line labels Lx/Rx refer to the pseudocode in §3 and to the matching comments in `solver/algorithm.py`.

4.1 Level 1 — Feasibility guarantee

By §1, deciding feasibility is NP-complete. A polynomial-time algorithm that *always* finds a feasible assignment whenever one exists would therefore show P = NP. We do not claim that. We prove the strongest statements a polynomial algorithm can make: (a) SSD-R never outputs an invalid schedule; (b) SSD-R never wrongly reports infeasibility; and (c) SSD-R is complete on an explicit, practically relevant class.

Theorem 4 (Soundness)**.** *If SSD-R returns with an empty reason, then σ satisfies F1, F2 and F3.*

Proof. We show the invariant *(I): the partial assignment held in the state satisfies F1, F2 and F3 restricted to placed tasks.* It holds initially, since nothing is placed (L5). We check every line that writes σ.

- **L10** places i^* in some $s \in F(i^*)$, computed at L7 against the *current* state. Membership in $F(i^*)$ means three things: $s \in [l_{i^*}, u_{i^*}]$, which is F3; the counter $\texttt{conf}[i^*][s] = 0$, so no placed neighbour is in s, which is F1; and $\text{load}(s) + r_{i^*} \le C(s)$, which is F2. The forward check only shrinks the candidate set to a subset of $F(i^*)$.

- **L15** moves an evicted task j only to $t \in F(j)$ (same argument). **L16** places i in s only after re-checking $s \in F(i)$ (`can_place`). If any eviction fails, every move is undone, which restores the previous state, and that state satisfies (I).

- **L19** applies a move only if $\texttt{can_place}(i, t)$ holds.

- **L20** applies a swap only if i can go to $\sigma(j)$ with j ignored, and j can go to $\sigma(i)$ with i ignored. Ignoring the partner is exact: after the swap the partner is no longer in that slot, so its load and its conflict no longer count there.

- **R2** starts each pass from an empty state.

An empty reason is returned only after L6 exits, that is, when every task is placed. By (I), σ is then feasible. As a second, independent line of defence, `run.py` re-validates every reported schedule with `solver/validate.py`. It found no violation on any feasible run in §6: 823 small random, 3 graded and 19 supplementary. □

Lemma 5 (Certificates)**.** *If Stage 1 returns* PROVEN_INFEASIBLE*, no feasible assignment exists.*

Proof. Suppose a feasible σ exists, and take each check in turn.

- **L1:** $\sigma(t_i)$ is a slot in the window of t_i with $r_i \le C(\sigma(t_i))$ (F3 and F2). So the L1 condition fails.

- **L3:** for an interval $[a, b]$, every $i \in S$ has $\sigma(t_i) \in [a, b]$ by F3. Summing F2 over $s = a..b$ gives $\sum_{i \in S} r_{ik} \le \sum_{s \in [a,b]} \text{load}_k(s) \le \sum_{s=a}^{b} C_k(s)$.

- **L4:** σ restricted to S is a proper colouring of $G[S]$ with colours $[a, b]$ (F1 and F3). Width 1 would then force $G[S]$ to have no edge. Width 2 forces it to be bipartite, and the BFS returns an odd cycle only if it is not. For width $w \ge 3$, a clique $Q \subseteq S$ needs $|Q|$ distinct colours, so $|Q| \le w$.

Each certificate we report (task, interval, cycle or clique) is checked in `tests/test_algorithm.py`. On 316 certified small instances the exact solver agreed every time. □

Theorem 6 (Completeness on a class)**.** *Suppose that (i) every task has fewer conflict neighbours than its window has slots,* $\deg(t_i) < u_i - l_i + 1$*, and (ii) capacity never binds:* $\sum_{all\ i} r_i \le C(s)$ *for every slot s. Then the first construction pass (R2 with $\Pi = [\,]$) places every task without calling* REPAIR*, and SSD-R returns a feasible σ.*

Proof. At L7, a slot s of t_i's window can be missing from $F(i)$ for only two reasons: a neighbour already sits in s, or s lacks capacity. By (ii), capacity never fails. Neighbours occupy at most $\deg(t_i)$ distinct slots. So $|F(i)| \ge (u_i - l_i + 1) - \deg(t_i) \ge 1$, the test at L9 never fires, and L6 terminates with all n tasks placed. Theorem 4 gives feasibility. □

This class contains the sparse, wide-window regime of the supplementary benchmarks, and a randomised test (200 instances) asserts that no restart or repair is ever triggered on it. Outside the class, SSD-R can fail. On 1200 small random instances it missed 1 of 824 feasible ones (§6), and that miss is reported as NOT_FOUND, never as infeasible.

4.2 Level 2 — Approximation ratio

Theorem 4 only uses window-respecting placements. Combined with Lemma 3 (§2), this gives a ratio. For an instance I, define

$$\rho(I) = \max_{i:\, w_i>0} \frac{u_i+1}{l_i+1} \;\le K, \qquad \alpha_P(I) = \max_{i:\, w_i>0} \frac{u_i+1+\lambda_R+\lambda_B/4}{l_i+1} \;\le K+1.25.$$

Theorem 7 (Ratio)**.** *Whenever SSD-R returns a feasible σ,*

$$P_{base}(\sigma) \le \rho(I)\cdot P_{base}(\sigma^*_{base}) \qquad \textit{and} \qquad P(\sigma) \le \alpha_P(I)\cdot P(\sigma^*),$$

where σ^*_{base} and σ^* are optimal for P_{base} and P respectively.

Proof. Upper bound on SSD-R. Every placement line (L10, L15/L16, L19, L20) keeps $\sigma(t_i) \le u_i$ (Theorem 4, F3). Hence $P_{\text{base}}(\sigma) \le \sum_i w_i(u_i+1)$.

Lower bound on OPT. Every feasible assignment has $\sigma^*(t_i) \ge l_i$, so $P_{\text{base}}(\sigma^*_{\text{base}}) \ge \sum_i w_i(l_i+1) > 0$.

Ratio of sums. Terms with $w_i = 0$ vanish on both sides. For the rest, $\frac{\sum_i a_i}{\sum_i b_i} \le \max_i \frac{a_i}{b_i}$ when all $b_i > 0$ (the mediant inequality). This gives $\rho(I)$, and $\rho(I) \le K$ because $u_i \le K-1$ and $l_i \ge 0$.

Full P. By the proof of Lemma 3, $R(\sigma) \le W$ and $B(\sigma) \le W/4$ for any feasible σ. So $P(\sigma) \le \sum_i w_i\,(u_i+1+\lambda_R+\lambda_B/4)$, and $P(\sigma^*) \ge P_{\text{base}}(\sigma^*) \ge \sum_i w_i(l_i+1)$. The mediant inequality again gives $\alpha_P(I)$. □

What the bound does and does not use. The proof relies only on the window restriction that every placement line enforces. It gives the greedy (L8, L10) and the local search (L19, L20) no credit. The tight example below shows that this is not slack in the analysis: those stages *cannot* improve the worst case. Empirically the picture is much better. The mean ratio is 1.054 and the maximum observed is 1.64 (§6). On the three small benchmarks, $\rho = K$: some task always has the window $[0, K-1]$. The intermediate quantity from the proof, $\sum w_i(u_i+1)/\sum w_i(l_i+1)$, is an instance-specific bound that is already tighter: 1.97, 1.94 and 1.90. The measured ratios, 1.14, 1.00 and 1.00, are well below it.

4.3 Level 3 — Tight adversarial example

Family I_K ($K \ge 2$), in `instances/tight_K{2,4}.json`. Tasks H (weight 1), Y (weight 0) and $Z_0, \ldots, Z_{K-2}$ (weight 0). All resources are 0, capacities are uniform, and every window is $[0, K-1]$. Conflicts: the Z_j form a clique, and every Z_j conflicts with H and with Y. There is no edge between H and Y.

Optimum: $H \to 0$, $Y \to 0$, $Z_j \to j{+}1$. Then $P_{\text{base}}(\sigma^*) = 1$ and $\rho(I_K) = \frac{K}{1} = K$.

Trace of SSD-R, citing pseudocode lines:

1. **Stage 1 passes.** Demand is 0 (L3). The only interval containing any task is $[0, K-1]$, of width K. The largest cliques, $\{Z_j\} \cup \{H\}$ and $\{Z_j\} \cup \{Y\}$, have size K, which is not more than K (L4).

2. **L8 picks the Z_j first.** Initially every $|F| = K$. The degrees are $\deg Z_j = K$ (that is, $K{-}2$ other Z's plus H and Y), while $\deg H = \deg Y = K-1$. After $Z_0, \ldots, Z_{j-1}$ are placed, Z_j, H and Y all have $|F| = K-j$. The degree tie-break still selects Z_j.

3. **L10 puts Z_j in slot j.** All the Z tasks have weight 0 and zero resources, so every candidate slot costs 0. The tie rule picks the lowest free slot. The forward check never fires, because H and Y keep at least two options until Z_{K-2} is placed.

4. **Then $H \to K{-}1$ and $Y \to K{-}1$.** Slots $0..K{-}2$ hold their neighbours, and H and Y do not conflict.

5. **L19 finds no improving move.** H cannot move to any $s \leq K-2$, because Z_s is there. Moving Y or a Z_j changes P by 0, so the move is not accepted.

6. **L20 finds no improving swap.** Swapping H with Z_j sends Z_j to slot $K-1$, where it conflicts with Y. Swapping Y with Z_j sends Z_j next to H. Swaps among the Z's, or between H and Y, have $\Delta P = 0$.

So $P_{\text{base}}(\sigma) = 1 \cdot K = K = \rho(I_K) \cdot P_{\text{base}}(\sigma^*)$: **the bound is attained exactly**, for every K. `tools/ratio.py instances/tight_K4.json` prints `ratio 4.000000000`, and `tests/test_algorithm.py` asserts the ratio is exactly K for $K = 2$ and $K = 4$.

For the full P, the same instance gives $P(\sigma) = K + \lambda_R \cdot 1 = K + 1$ against $P(\sigma^*) = 1$. The bound is $\alpha_P = K + 1.25$. The remaining gap of $\lambda_B/4$ exists because, with zero resources, the imbalance term is 0 in both solutions.

Positive weights. The generator draws $w \in [1, 10]$. If the zero-weight tasks get weight $\varepsilon > 0$, the ratio becomes $\frac{K+\varepsilon(\cdot)}{1+\varepsilon(\cdot)} \to K$ as $\varepsilon \to 0$. So ρ is also the best constant for strictly positive weights, approached as closely as desired. The specification's $w : V \to \mathbb{R}$ admits the zero-weight instance, where the bound is attained exactly.

Why only a different approach helps. To reach σ^*, the algorithm must move H, Y and some Z_j *at the same time*. That is a 3-task Kempe-chain interchange of slots j and $K-1$, which lies outside the move-or-swap neighbourhood. Escaping needs a larger neighbourhood, such as Kempe chains or an exact search on small components. That is a different algorithm, and it would also need a different tight example. The same 3-task trap explains the 1.141 ratio on benchmark seed 1 (§6, A2).

5 Task 5: Implementation

Language and dependencies. Python 3.10 or later (developed on 3.12). The solver, the CLI and the tests use *only the standard library*; numpy and matplotlib are used only by the chart script. None of the forbidden libraries is used. This command returns nothing:

```
grep -rnE "^\s*(import|from)\s+(ortools|pulp|cplex|gurobi|z3|networkx|pysat|pycosat)" \
     solver/ tools/ tests/ run.py
```

All algorithmic code is original. The generator (`generator.py`) is byte-identical to the specification, and a unit test checks its SHA-256 hash.

5.1 Layout

file	lines	role
`solver/instance.py`	98	data classes, JSON I/O, shape checks for any d and per-slot C
`solver/validate.py`	80	independent F1/F2/F3 checker and P_{base}
`solver/penalty.py`	80	$P = P_{\text{base}} + R + B$ (§2)
`solver/algorithm.py`	489	SSD-R (§3)
`solver/bruteforce.py`	101	exact branch-and-bound, used as ground truth
`run.py`	93	CLI
`tools/bench.py`	187	§6 tables and figures
`tools/reduction.py`	79	§1 construction and machine check
`tools/ratio.py`	30	SSD-R vs optimum on one file (§4)

5.2 Interface

```
python run.py --n 50 --K 8 --density 0.25 --seed 10   # generate, then solve
python run.py --input instances/toy.json --trace      # JSON file, step log on stderr
python run.py ... --solver brute --out result.json     # exact solver, write file
```

The input is the generator's dictionary saved as JSON. The output has exactly the five required keys: `assignment` (task id → 0-based slot), `penalty` (P, float), `runtime_ms` (int), `feasible` (bool) and `violation_reason`. The reason starts with `PROVEN_INFEASIBLE:` followed by a certificate, or with `NOT_FOUND:` followed by the task the heuristic could not place. In the `NOT_FOUND` case, `assignment` holds the valid partial assignment. If a schedule ever failed re-validation, it would be reported as `feasible=false` with `INVALID_OUTPUT`, never silently emitted.

5.3 Pseudocode-to-code map

Every label L1–L21 and R1–R3 appears as a comment on the matching line of `solver/algorithm.py`.

pseudocode	code
Stage 1 (L1–L4)	`certify_infeasible`, `edge_in`, `odd_cycle`, `largest_greedy_clique`
R1–R3	`solve`
L5–L11	`construct`; forward check in `takes_last_option`
L12–L17	`repair`
L18–L20	`local_search`
$F(i)$, $c(i,s)$, ΔB	`State.feasible_slots`, `State.task_cost`, `State.imbalance_delta`

Data structures. `State` keeps:

- the slot loads;
- a counter `conf`$[i][s]$ of t_i's neighbours placed in slot s, so an F1 check is $O(1)$ rather than $O(\deg)$;
- running sums $\sum u$ and $\sum u^2$ for each dimension, so ΔB costs $O(d)$ rather than $O(Kd)$.

The resource dimension d and the per-slot capacities are read from the input. No code assumes $d = 4$ or uniform capacities.

5.4 Tests

There are 49 `unittest` tests (`python -m unittest discover -s tests`, about 1.6 s). The four the specification requires:

required case	test (`tests/test_algorithm.py`)
all-conflict graph, $\chi > K$	K_{K+1} for $K = 2, 3, 5 \Rightarrow$ `PROVEN_INFEASIBLE`; K_K is solved
zero-capacity slot	slot never used; if it is a task's only slot ⇒ `PROVEN_INFEASIBLE` (L1)
tight SLA windows	width-1 windows pin the unique schedule; clashing pins are proven infeasible; width-2 windows with capacity for one task per slot ⇒ the heavier task goes first
single task	placed at l_i with $P_{\text{base}} = w_i(l_i + 1)$

Other tests:

- a 5th resource dimension, and unequal slot capacities;
- soundness against the exact solver on 120 random instances: valid output, no false certificate, never better than OPT;
- `NOT_FOUND` partial assignments are valid;
- the odd-cycle and clique certificates are genuine;
- the tight example gives a ratio of exactly K;
- the completeness class of Theorem 6 (200 random instances);

- the reduction of §1;
- hand-computed values for P and its bounds;
- the branch-and-bound solver against plain enumeration;
- the generator hash, and the CLI's five output keys.

5.5 Docstrings

Every non-trivial function has a docstring that explains *why* it is designed as it is. Examples: why `odd_cycle` is exact for width 2; why `repair` goes only one level deep; why the forward check exists, including the instance that motivated it; why zero-capacity cells are left out of B; and why the branch-and-bound lower bound is valid for P.

6 Task 6: Empirical Analysis and Benchmarking

Setup. Python 3.12 on one core of an ARM64 Linux VM. Every row runs through `run.solve`, the same code path as `python run.py ...`. `tools/bench.py` regenerates every number and figure in this section; its raw output is in `report/data/`. Runtime is the median of 3 runs. "OPT" is the exact optimum of P from the branch-and-bound solver (`solver/bruteforce.py`). That solver is cross-checked against plain K^n enumeration in `tests/test_validate.py` and `tests/test_penalty.py`.

6.1 The nine graded benchmarks

n	K	dens.	seed	$\lvert E\rvert$	status	$P(\sigma)$	ms	OPT	ratio
8	3	0.30	1	11	feasible	81.378	0.49	71.308	1.141
10	4	0.40	2	16	feasible	78.020	0.70	78.020	1.000
12	4	0.50	3	26	feasible	135.269	0.97	135.269	1.000
50	8	0.25	10	291	proven infeasible	–	3.6	–	–
100	10	0.30	11	1499	proven infeasible	–	19.1	–	–
150	12	0.35	12	3942	proven infeasible	–	9.3	–	–
200	15	0.40	20	7936	proven infeasible	–	27.1	–	–
200	5	0.60	21	11904	proven infeasible	–	2.6	–	–
200	20	0.10	22	1974	proven infeasible	–	256.1	–	–

Table 1: SSD-R on the graded suite. P_{base} of the three feasible runs: 69.77, 68.61, 123.56. The CLI's integer `runtime_ms` is 0 for the small rows. The table shows sub-millisecond wall time.

Feasibility. Every `feasible=true` output passes the independent validator (`solver/validate.py`). The exact solver agrees with every verdict it can check: OPT exists for $n \le 12$, and branch-and-bound confirms nothing beyond what SSD-R reports.

6.2 Anomalies

A1 — six of nine benchmarks are infeasible. This is the main finding, and it is a property of the provided generator, not a failure of SSD-R. Each `violation_reason` is a certificate that a reader can check by hand:

- **seed 10** (n=50): the triangle $T3$–$T8$–$T23$ has every window equal to $[3, 4]$. Three mutually conflicting tasks cannot share 2 slots.
- **seed 11** (n=100): the 4-clique $\{T13, T14, T25, T59\}$ is confined to the 3 slots $[7, 9]$.

- **seed 12** (n=150) and **seed 20** (n=200, K=15): network demand is $82.8 > 72$ and $94.6 > 90$ Gbps over all slots. This is *structural*. The generator draws $r_k \sim U(1, \lfloor \text{cap}_k/(\lfloor n/K \rfloor + 1) \rfloor)$. When that floor is 0 ($\lfloor 6/13 \rfloor = \lfloor 6/14 \rfloor = 0$), Python's `uniform(1,0)` returns values in $[0, 1]$ with mean 0.5. Expected Net demand is then $0.5n$, against $6K$ of capacity: $75 > 72$ and $100 > 90$. We confirmed that these two (n, K) pairs are infeasible at every density in $\{0.02, 0.05, 0.1\}$ for seeds 0–4.

- **seed 21** (n=200, K=5): the triangle $T10$–$T67$–$T73$ lies inside $[0, 1]$. The same instance also fails capacity in two dimensions, GPU $103.8 > 40$ and Net $98.2 > 30$; Stage 1 reports the first certificate it finds.

- **seed 22** (n=200, K=20): the 5-cycle $T102$–$T170$–$T57$–$T198$–$T63$ lies inside $[18, 19]$. An odd cycle needs 3 colours, and only 2 slots are available. A single greedy clique pass *missed* this case: the only triangle in it, $\{T63, T102, T198\}$, was not found because the greedy started from the highest-degree vertex. This led us to make the width-2 check an exact bipartiteness test (L4).

Why so many? 31% of all generator windows have width 2, and 50% have width ≤ 3. If t tasks are confined to a 2-slot window, they contain a triangle with probability roughly $1 - (1 - p^3)^{\binom{t}{3}}$ (ignoring dependence between triangles). At $p = 0.25$ this is 0.27 for $t = 6$ and 0.59 for $t = 8$. In seed 10, windows of exactly width 2 hold 2–7 tasks each (7 in $[6, 7]$). There are $K - 1$ such windows, and the odd-cycle test also catches cycles longer than 3, so the probability that *some* window fails is high. The certificates come from Stage 1 alone, in 2.6–256 ms. Without them, a heuristic would either search the 6 instances until it timed out or report "not found" with no reason.

A2 — seed 1 ratio 1.141. SSD-R puts $T3$ (w=2.80) in slot 0, and $T5$ (w=5.94) and $T6$ (w=4.64) in slot 1. The optimum swaps them: $T3 \to 1$, $T5, T6 \to 0$. $T3$ conflicts with *both* $T5$ and $T6$, so every single move and every pairwise swap is infeasible. Swapping $T3$ with $T5$ puts $T3$ next to $T6$. Reaching the optimum needs a simultaneous 3-task exchange, which is a Kempe-chain interchange of slots 0 and 1 on the component $\{T3, T5, T6\}$. This is the same structure as our tight example (§4) on a real benchmark: L8 commits the conflict hub $T3$ (degree 4, the highest) first, and L19–L20 cannot undo it.

A3 — runtime is not monotone in n. The infeasible rows are fast, 2.6 ms at n=200 for seed 21, because Stage 1 returns as soon as the first certificate is found. Intervals are scanned from $[0, 0]$ upward, so a certificate in a low window is found early. Seed 22 takes 256 ms because its certificate is in the last window, $[18, 19]$, so all $O(K^2)$ intervals, each with a clique search, run first.

6.3 Scaling on feasible instances

None of the graded instances above n=12 is feasible, so they cannot show how the constructive and improvement stages scale. We therefore ran 20 supplementary instances, with density 0.05 and seeds 0–4, for $(n, K) \in \{(25, 6), (50, 8), (100, 10), (200, 20)\}$. 19 are feasible and solved. One, $(100, 10)$ seed 2, is proven infeasible by the *sub-interval* demand check: tasks confined to $[1, 9]$ need 54.95 Gbps against 54.0. The global check over $[0, 9]$ passes, which is why L3 scans all intervals.

n	solved	mean P	mean ms
25	5/5	457.5	5.8
50	5/5	1193.6	26.9
100	4/4	3071.0	118.6
200	5/5	11317.4	768.5

Penalty grows roughly as n^2/K. P_{base} sums $w_i(\sigma_i + 1)$, and with K growing alongside n the average slot index also grows. **Runtime** on feasible instances fits $t \propto n^{2.33}$ on a log-log scale. This

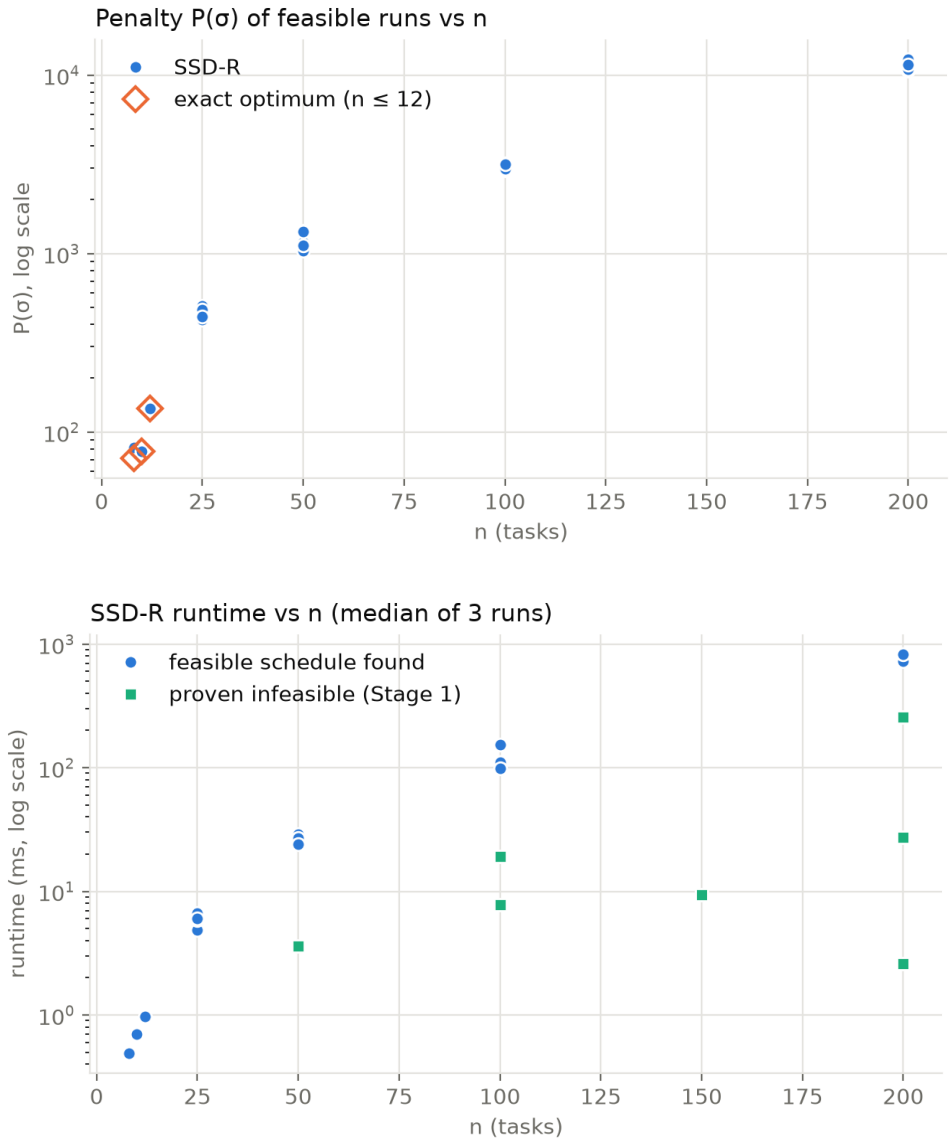


Figure 1: Top: $P(\sigma)$ against n for every feasible run (log scale). Orange diamonds are the exact optima for $n \leq 12$. Bottom: runtime against n (log scale). Blue circles are feasible runs; green squares are runs ended by a Stage 1 certificate.

is below the worst-case $O(n^3Kd)$ of §3, because restarts are rare (none were needed on these 19 instances), so the practical cost is one constructive pass ($O(n^2Kd)$) plus local search ($O(n^2d)$ per pass).

6.4 Approximation quality vs the exact optimum

To go beyond three small data points, we ran SSD-R and the exact solver on 1200 random small instances: $(n, K, \text{dens}) \in \{(6,3,.3),(8,3,.3),(8,4,.4),(10,4,.4),(10,3,.2),(12,4,.5)\}$, seeds 0–199 (`report/data/small_study.csv`).

	exact: feasible	exact: infeasible
SSD-R feasible	823	0
SSD-R proven infeasible	0	316
SSD-R not found	1	60

- **Soundness:** no false certificates (0 in the middle-left cell: SSD-R never proved a feasible instance infeasible) and no wrong feasibility claims (0 in the top-right cell).
- **Completeness of Stage 1:** 316 of 376 infeasible instances (84%) are certified. The other 60 need a clique larger than the greedy finds, or a colouring argument that is not a clique. They are honestly reported as `NOT_FOUND`.
- **Missed feasible:** 1 in 824 (0.12%), seed 171 of $(12, 4, 0.5)$. Its exact solution is essentially a unique 4-colouring of a dense graph, with six tasks restricted to 2–3 slot windows. The restart promotions cycle through six tasks ($T4 \to T3 \to T11 \to T8 \to T6 \to T10 \to T4 \ldots$), and each promoted task displaces the next. During development this count was 58, and two fixes (§3) reduced it. Restarts (R1–R3) brought it to 2. Forward checking (L10) brought it to 1; it fixed the restart cycle on $(100, 10, 0.05)$ seed 1, where four tasks with nested windows evicted each other in turn.
- **Ratio** over the 823 jointly feasible instances: mean 1.054, median 1.000, 95th percentile 1.242, maximum 1.644. 59.1% are solved exactly.

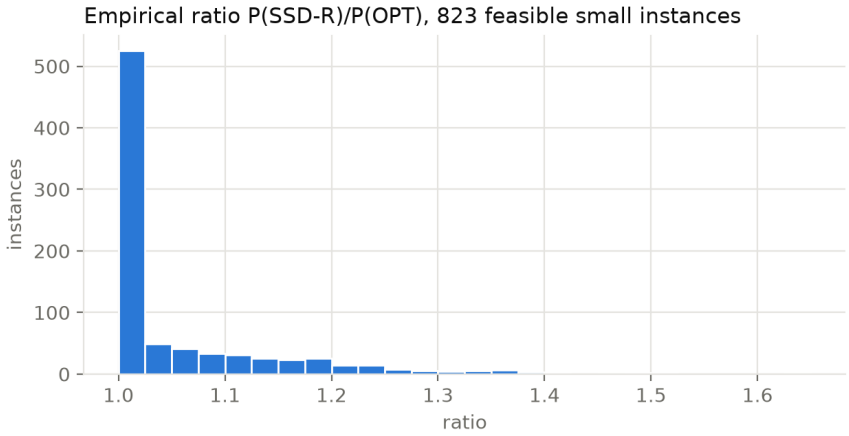


Figure 2: Distribution of $P(\sigma_{\text{SSD-R}})/P(\sigma^*)$ over 823 feasible small instances.

7 Task 7: Design Journal

7.1 The hardest design decision: what Stage 1 is allowed to claim

The hardest decision was in Stage 1, and it was not about speed. It was about *what my algorithm is allowed to say.* My first version had a single greedy clique check (largest degree first) for every interval. When I ran it on benchmark seed 22 (n=200, K=20), SSD-R placed 6 tasks and then returned `NOT_FOUND` on $T63$. At that point I had to choose. One option was to strengthen the search, with more restarts or a deeper repair, and treat the instance as hard. The other was to treat it as possibly impossible and strengthen the *proof.*

Looking at the tasks confined to window $[18, 19]$ settled it. $T63$, $T102$ and $T198$ form a triangle, and my greedy clique, starting from the highest-degree vertex, had walked past it. No amount of search could ever have succeeded. The trade-off was between a general but incomplete certificate (cliques for every width) and an exact but narrow one. I chose both. For width-2 intervals, colourability is exactly bipartiteness, which BFS decides in linear time. So I made that case exact (an odd-cycle certificate), and kept the clique heuristic only for width ≥ 3. The rejected alternative was a small exact colouring search on each interval. It is more complete, but it is exponential in the worst case, and it would blur the clean line I wanted: Stage 1 is polynomial and never wrong, Stages 2–4 are polynomial and may give up. The deciding fact was the data: 31% of generator

windows have width 2, so the exact case covers the most common situation. After the change, all six large benchmarks come with a certificate a reviewer can check by hand.

7.2 Where SSD-R failed empirically, and the next week

Failure 1 — benchmark $(8, 3, 0.3)$**, seed 1, ratio 1.141.** SSD-R puts the conflict hub $T3$ (degree 4, weight 2.8) in slot 0, and pushes $T5$ (5.94) and $T6$ (4.64) to slot 1. The optimum swaps them, but $T3$ conflicts with both, so no single move or pairwise swap is feasible. The local search is stuck in a 3-task trap. My own tight example (§4) has the same structure; I had built it on paper before seeing it in a benchmark.

Failure 2 — random instance $(12, 4, 0.5)$**, seed 171:** `NOT_FOUND` **on a feasible instance.** The squeaky-wheel restarts cycle: $T4 \to T3 \to T11 \to T8 \to T6 \to T10 \to T4$. Each promoted task takes its cheapest slot, and that evicts the next one. Earlier in development, the same failure mode cost 58 of 824 feasible instances. Restarts brought that to 2, and forward checking at L10 brought it to 1. This one survives because its solution is essentially a unique 4-colouring.

With one more week I would add *Kempe-chain moves* to Stage 4: swap two slots on a whole connected component of the two-slot conflict subgraph. That move fixes Failure 1 directly. Its catch is that it would also escape my current tight example, so I would need to find a new one and re-prove the tightness level. For Failure 2, I would add an exact backtracking fallback restricted to the tasks involved in a restart cycle. It is exponential only in that small set, and it returns to the greedy afterwards.

7.3 Where this appears at ScoreMe

The closest match is the **OCR GPU cluster fed by a Kafka consumer group**. Bank-statement OCR jobs arrive on a Kafka topic, and each partition is consumed by one consumer per execution cycle. Mapped onto our problem:

- *Slots* are the 30-second scheduling cycles of the GPU pool.
- *Resources* are GPU units (OCR models pinned to cards), CPU cores for pre-processing, RAM for page images, and network bandwidth for pulling statements from object storage.
- *Conflicts* are two jobs that write to the same Kafka partition, where ordering must hold, or that share a GPU memory bus.
- *SLA windows* come from the lender's turnaround commitment. A statement uploaded at T=0 for a Tier-1 bank must be parsed within k cycles.

SSD-R would run once per planning horizon, for example every 10 cycles, over the queued jobs. Stage 1 is the most valuable part operationally. If the queued work *cannot* meet its SLAs, it reports exactly which jobs collide and where: "these 5 statements form an odd conflict cycle inside a 2-cycle window", or "Net demand over cycles 1–9 exceeds the link". That is actionable. The platform team can add a GPU node, re-partition the topic, or tell the lender before the SLA breaches, not after. The R term of P keeps jobs away from their SLA edge, so that a single OCR retry does not cause a breach. The B term stops one cycle from saturating the GPUs while the next sits idle.

7.4 What surprised me

- **The provided benchmarks are mostly infeasible.** Six of the nine are. Two of them are infeasible for a reason that has nothing to do with scheduling: the generator calls `uniform(1, cap//(n//K+1))`, and for n=150, K=12 that upper limit is $6//13 = 0$. Python then silently samples from $[0, 1]$, and Net demand ends up above capacity at every conflict density. I expected the stress instances to stress the heuristic. They stress the infeasibility detector instead.

- **Proving "impossible" quickly was more valuable than optimising.** Most of my effort in the end went into Stage 1 and the failure modes, not into the objective.
- **The approximation bound gives the clever parts no credit.** The ratio comes entirely from *feasibility* (every task stays inside its window). The greedy and the local search contribute nothing to the worst case. I only believed this once the tight example showed SSD-R hitting the bound exactly, and once the same trap then appeared on a real benchmark.
- **About my own thinking:** my first instinct with each failure was to add search. Every real improvement came from first asking whether the instance was solvable at all, and checking that against the exact solver.

8 Task 8: Viva Preparation Notes

8.1 Hand trace: toy instance

Using 0-based slots, the toy instance's windows are $T1$:$[0,2]$, $T2$:$[0,3]$, $T3$:$[0,3]$, $T4$:$[1,3]$, $T5$:$[0,3]$ and $T6$:$[1,3]$. Stage 1 passes: total demand is within capacity, and the conflict graph is the 6-cycle $T1$–$T2$–$T4$–$T6$–$T5$–$T3$–$T1$. That cycle is even, so the largest clique is 2 and no odd cycle exists. Stage 2 then runs as follows (`python run.py --input instances/toy.json --trace`). Every task has degree 2, so ties on $|F|$ are broken by weight.

step	chosen (L8)	why	$F \to$ slot (L10)
1	$T1$	$\|F\| = 3$, tied with $T4, T6$; heaviest (w=5)	$\{0,1,2\} \to 0$
2	$T2$	$\|F\| = 3$, tied with $T3, T4, T6$; heaviest (w=4)	$\{1,2,3\} \to 1$
3	$T4$	$\|F\| = 2$ ($T2$ blocks slot 1)	$\{2,3\} \to 2$
4	$T6$	$\|F\| = 2$ ($T4$ blocks slot 2)	$\{1,3\} \to 1$
5	$T3$	$\|F\| = 2$: $T1$ blocks 0, and slot 1 Net $3 + 1.5 + 2 > 6$ (F2)	$\{2,3\} \to 2$
6	$T5$	only task left; $T6$ blocks 1, $T3$ blocks 2	$\{0,3\} \to 0$

L19 and L20 find no improving step. The result is $P = 38.53$, against the exact optimum $P^* = 37.90$ (ratio 1.017). The optimum puts $T1, T4, T5$ together in slot 1 (32 CPU and 8 GPU, exactly at capacity). Practise this trace until it matches the `--trace` output line by line. Also practise one fresh `generate_instance(6, 3, seed=...)` instance, checked with `--trace` and `--solver brute`.

8.2 Prepared answers

"What if I add a 5th resource dimension?" Nothing in the code changes. d is read from the input (`Instance.d`). Every capacity check loops over $k < d$, and B averages over d dimensions. Costs grow linearly in d, from $O(n^2Kd)$ upward. Proofs: Lemma 5 and Theorem 4 are per dimension, so they still hold. The bound $B \leq W/4$ is unchanged, because it is an average. One effect: an extra binding dimension can only shrink $F(i)$, so there are more restarts and more `NOT_FOUND`s. The test `test_five_dimensions` shows the 5th dimension deciding the placement.

"What if two slots have different capacities?" Already supported. Capacities are stored per slot, $C(s)$, and every F2 check and the L3 interval sum use $C(s)$. B uses utilisation load/$C(s)$, so a half-size slot at 50% load counts as 50%, not 25%, and zero-capacity cells are excluded. The test `test_unequal_capacities` covers it. One subtlety: L1 ("task fits nowhere") becomes more powerful, because a large task may fit only in the big slots.

"A decision you would make differently in hindsight?" I would build the local search around Kempe-chain interchanges from the start, rather than single moves and pairwise swaps. Both the benchmark ratio of 1.141 and the tight example are 3-task traps that a Kempe move escapes. I chose move and swap because they are simple to prove sound and cheap, but they are exactly the neighbourhood the adversary exploits.

Things to know cold:

- why slots are 0-based with $\sigma + 1$ in the penalty;
- why the width-2 check is exact;
- why every certificate is sound (Lemma 5);
- the order key at L8: $(\text{promoted}, |F|, -\deg, -w, i)$;
- why the forward check exists;
- why α needs no credit from the greedy;
- how the tight family traps moves and swaps;
- the generator's `uniform(1,0)` effect.